

```

Integración (scipy.integrate): quad(f, a, b):  $\int_a^b f(x) dx$ , dblquad(f, a, b, gfun, hfun):
 $\int_a^b \int_{g(x)}^{h(x)} f(x, y) dy dx$ , simpson(y, x) (método de Simpson, datos discretos).
EDOs: solve_ivp(fun, [t0, tf], y0, method='RK45'): Sistema  $\dot{y} = f(t, \vec{y})$ , odeint(f, y0, t): legacy.
Optimización (scipy.optimize): minimize(fun, x0), root(fun, x0) (ceros de función),
curve_fit(f, xdata, ydata) (mínimos cuadrados no lineales)
Interpolación (scipy.interpolate): interpnd(x, y, kind='cubic'), griddata((x, y), z, (X, Y)).
Fourier (scipy.fft): fft(x) (Rápida), fftfreq(n, d).
Constantes (scipy.constants): c, h, hbar, G, e, m_e, g, ...
MATPLOTLIB (pyplot: plt) – cheatsheet
Core: fig, ax = plt.subplots(nrows, ncols, figsize=(w, h)).
Traza: ax.plot(x, y, 'r--', label=' $\$plot(x)\$'$ , lw=2).
Scatter: ax.scatter(x, y, c=z, cmap='viridis', s=10).
Estadística: ax.bar(x, h), ax.hist(data, bins=n), ax.pie(v).
Campos: ax.quiver(X, Y, U, V) (vectores), ax.streamplot(X, Y, U, V).
Mapas: im=ax.imshow(h, origin='lower', extent=(X0,xf,y0,yf)), plt.colorbar(im, ax=ax).
Contornos: ax.contour(X, Y, z, lev), ax.contourf (relleno).
Formato: set_xlabel('x'$phi$'), set_xlim(min, max), set_yscale('log'), legend(loc='best'), grid(linestyle=':')
3D: ax = fig.add_subplot(projection='3d'), plt.surface(X, Y, Z).
Salida: plt.savefig('plot.pdf', dpi=300, bbox_inches='tight'), plt.show(), plt.close().

```

SYMPY (sym

```

Vars: x, y, z = symbols('x y z'), init_printing().
Ops: simplify(expr), expand(e), factor(e), collect(e, x).
Cálculo: diff(f, x, n) ( $\partial_x^n f$ ), integrate(f, (x, a, b)), limit(f, x, oo) ( $\lim_{x \rightarrow \infty} f$ )
Solve: solveset(eq, x) (algebraico), dsolve(eq, f(x)) (EDO simbólica).
Matrices: Matrix([[1,0],[0,1]]), M.det(), M.inv().
Numérico: N(expr, digits), f_np = lambdify(x, expr, 'numpy').
PANDAS (pd)
Estructuras: Series(data, index), DataFrame(data, columns).
I/O: read_csv/excel/sql('path'), to_csv('path').
Info: df.head(), df.info(), df.shape, df.describe(), df.dtypes, df.index, df.columns.
Selección: df['col'], df[['a', 'b']] (dataframe), df.iloc[fila, col], df.loc[idx, 'col'] (etiqueta).
Filtrado: df[df['E'] > 0], df.query('E > 0 & P < 1').
Limpieza y Manipulación: df.dropna() (elimina nulos), df.drop_duplicates(), df.apply(func), df.fillna(v) (rellena), df.rename(columns={'a':"b"}), df.sort_values('c').
Agrupación: df.groupby('cat').agg(['mean', 'sum']).
Fusión: merge(d1, d2, on='k'), concat([d1, d2]).
Series Temporales: to_datetime(df['t']), df.set_index('t')
PRECISIÓN ARBITRARIA (mpmath)

```

ptable.

Configuración: mp.dps = 50 (50 decimales).
Tipos: mpf(1) / 3, mpc(1j).

PROPAGACIÓN DE INCERTIDUMBRES (uncertainties)

Automatiza el cálculo de errores mediante derivadas parciales: $\sigma_f^2 \approx \sum (\partial_{x_i} f)^2 \sigma_{x_i}^2$.
Variables: x = ufloat(val, err).

Arrays: `numpy.ndarray([vals], [errs])`.
 Acceso: `x.n` (valor nominal), `x.s` (desviación estándar).

Funciones: `numpy.sin(x)`, `numpy.exp(x)`.

VISUALIZACIÓN ESTADÍSTICA (`seaborn: sns`)

Capa de alto nivel sobre Matplotlib orientada a la exploración de datos.
Estilo: `sns.set_theme(style="whitegrid")`

```
Distribución: sns.histplot(data, kde=True), sns.kdeplot(x).
Relacional: sns.scatterplot(x, y, hue="cat"), sns.lmplot()
```

Categorías: sns.boxplot(), sns.violinplot().

CONTROL DE VERSIONES EN VS CODE (GIT GUI)

Paradigma: *Source Control* (Ctrl+Shift+G): observar en tiempo real el estado del repositorio.
Changes: representa el *working directory*. *Staged Changes*: representa el *index*.

FLUJO DE TRABAJO LOCAL	
Operación Git	Acción en VS Code

git init	Botón Initialize Repository en el panel lateral.
----------	---

git status	Lista de archivos: M (Modified), U (Untracked) o A (Added).
git diff	Clic sobre el archivo: Abre el editor de diferencias (Δ).

git add	Clic en el icono + (Plus) junto al nombre del archivo o sección.
git commit	Escribir mensaje en el cuadro superior y clic en Commit (check).

git log Panel **Timeline** en la parte inferior del explorador de archivos.

Gestión en la Barra de Estado (esquina inferior izquierda) y el menú de contexto (...).

- **Crear y Cambio:** Clic en nombre rama actual → Create new branch o seleccionar existente.
- **Publicar:** Icono nube barra estado (*Publish Branch*). Automáticamente enlaza con remoto.
- **Fusión (Merge):** Menú → Branch → Merge Branch. Seleccionar la rama origen.

- **Fusion (Merge):** **Menu ... → Branch → Merge Branch...** Seleccionar la rama origen para integrar en la rama activa.

1. Abrir archivo en conflicto.
2. Clic en **Resuelve en Merge Editor** (esquina inferior derecha).

3. Interfaz de 3 paneles: *Current* (local), *Incoming* (remoto/otra rama) y *Result*.

- Stash (Memoria temporal): Menu ... → Stash → Stash (Include Untracked). Ideal para limpiar el área de trabajo antes de un *checkout* sin perder el progreso.

SINCRONIZACIÓN Y REMOTOS	
Acción	Procedimiento en UI

Push / Pull	Botón circular Sync Changes en la barra de estado (ejecuta ambos).
Fetch	Menú Fetch → Pull → Push → Fetch (actualiza los datos sin alterar archivos).

Fetch	Menu $\dots \rightarrow$ Pull, Push $\rightarrow$ Fetch (actualiza metadatos sin alterar archivos)
Remote Add	Panel <i>Source Control</i> $\rightarrow$ sección <i>Remotes</i> $\rightarrow$ Icono $+$.

GESTIÓN DE ERRORES (UNDO)
- **Descartar cambios:** Icono de flecha curva (*Discard Changes*) sobre el archivo. Elimina el Δ

- **Deshacer último commit:** Menú ... → Commit → Undo Last Commit.

- **Sacar de Staging:** Icono - (Minus) en la sección *Staged Changes*.

Author: [Jorge Acebes Hernandez](#). Code: [GitHub](#). Last revised: June 29, 2020. [CC BY-NC 4.0](#)